

Informe de la Crisis de DiplomadosOnline

El problema que presentan en DiplomadosOnline es un caso comun que toma por sorpresa y la respuesta operativa fue insuficiente para abarcar todo, ya que pasaron de manejar grupos pequeños de estudiantes a gestionar una gran cantidad de estudiantes de manera exponencial, pero siguieron usando la misma herramientas ofimaticas del primer día, el programa que conocemos como Excel, y todo el control lo llevan en un archivo compartido (Alumnos_y_Profesores_General.xlsx).

Este problema desencadenó como consecuencia tres graves problemas: Inconsistencia y Errores Humanos al depender del tipeo manual, esto llevo a Cédulas y datos duplicados, nombres mal escritos y asignaciones erróneas, y no hay manera de verificar que los datos se esten colocando correctamente. Vulnerabilidad de la Información ya que toda la empresa depende de un único archivo Excel y se debe tener precaucion de que dicho archivo no se llegue a corromper, que no se cierre sin guardar, o que no lo lleguen a borrar por accidente, si llegara a fallar el archivo al guardar se perderia una gran cantidad de informacion. La otra consecuencia seria la Parálisis Operativa en la Graduación, al cerrar cada uno de los periodos, se debe realizar una revision de miles de filas a mano, calcular promedios uno por uno, aplicando distintas reglas según el programa y copiar a mano a los graduados. Este metodo puede llegar a retrasar la entrega de diplomas hasta por semanas. Se intento mantener todo el seguimiento usando solo un archivo Excel, pero dicho archivo quedo corto para los requerimientos actuales

Requerimientos del MVP (Producto Mínimo Viable)

Para solucionar esto los integrantes de DiplomadosOnline, llegaron a la conclusion de desarrollar un Sistema de Gestión Académica (SGA-DO), para llevar un mejor control de los datos, y mi objetivo como desarrollador, es encargarme de construir el Motor Core (el Backend) de la aplicacion por Consola. Y para comparar la versatilidad en distintas opciones, se realizara con los lenguajes de programacion, Python, C++ y Java.

Requerimientos Funcionales

Para que sea considerado funcional y pueda resolver la crisis, se debe cumplir con estos requerimientos:

1. Gestión de Alumnos: Permitir el registro de alumnos (Cédula, Nombre, Correo, Programa) guardándolos en memoria RAM y sincronizándolos de inmediato en alumnos.txt (con notas iniciales en 0).
2. Gestión de Profesores: Permitir el registro de profesores (Cédula, Nombre, Correo, Especialidad, Materia) guardándolos en memoria y en profesores.txt.
3. Carga de Notas: Permitir buscar un alumno por Cédula e ingresarle hasta un máximo de 3 calificaciones.
4. Deshacer Nota: Registrar cada nota ingresada en una Pila (Stack). Permitir eliminar la última nota registrada al presionar la opción de deshacer (Ctrl+Z).
5. Procesamiento de Certificados: Evaluar de forma automatizada qué alumnos están aprobados según su tipo de programa, ingresarlos a una Cola en orden de llegada y exportar el reporte a certificados_pendientes.txt.
6. Reporte General: Leer los archivos de texto (alumnos.txt y profesores.txt) y mostrar en consola la lista ordenada de profesores activos y alumnos con su estatus.
- 7 Ciclo de Vida y Persistencia: Mantener la aplicación en un bucle continuo hasta que se elija "Salir", momento en el cual se asegura la persistencia en disco y se libera la memoria.

3. Plan Lógico de Trabajo

Para construir este MVP sin complicarnos ni enredarnos con el código, este es el plan de acción estructurado en 4 fases lógicas:

FASE 1: Modelo POO y Reglas de Negocio

- Crear clase Persona, Alumno y Profesor
- Crear ProgramaAcademico y subclases (Polimorfismo) |

FASE 2: Módulo de Persistencia (.txt)

- Cargar datos desde .txt a RAM al arrancar
- Escribir/Actualizar alumnos.txt y profesores.txt

FASE 3: Estructuras de Datos (Pila y Cola)

- Pila (Stack) para el módulo de Deshacer (Ctrl+Z)
- Cola (Queue) para el flujo de Certificaciones

FASE 4: Interfaz de Consola y Pruebas

- Bucle interactivo del menú (Opciones 1 a 7)
- Validación de bordes (Cédulas, máx 3 notas, etc.)

Detalle de la Ejecución:

Fase 1: Modelado de Clases (El Esqueleto)

Defino la jerarquía de Persona y ProgramaAcademico, y pruebo en un archivo borrador las 3 reglas de aprobación (en especial la del Bootcamp, que es la única que no usa promedio directo) para asegurarme de que el polimorfismo devuelva True o False correctamente.

Fase 2: Lectura y Escritura de Archivos (Persistencia)

Creo las funciones encargadas de pasar líneas de texto a objetos en memoria (usando `split(',')`), y aseguro que al arrancar el programa, la memoria se llene con lo que ya existe en `alumnos.txt` y `profesores.txt`.

Fase 3: Lógica de Pila y Cola

Pila: Cada vez que la Opción 3 añade una nota, guardo un puntero/referencia del alumno y el valor de la nota en el Stack. Si la Opción 4 se activa, hago `.pop()` y le quito esa nota al alumno. **Cola:** La Opción 5 filtra la lista de alumnos en memoria usando `.estaAprobado()`, mete los graduandos a la Queue y luego va desencolando uno a uno mientras escribe el reporte amigable en `certificados_pendientes.txt`.

Fase 4: Menú Interactivos y Validaciones

Monto el while del menú principal y Agrego validaciones simples: que no deje meter más de 3 notas por alumno, que avise si una cédula no existe y que no rompa el programa si la pila de deshacer está vacía.